UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 8

TEMPERATURE CONTROL SYSTEM

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 8

Instrumentation: Temperature Control Using Raspberry Pi

Objectives

- To interface a temperature sensor and a heating element with a Raspberry Pi.
- To implement a temperature control algorithm to maintain a target temperature.
- To log temperature data and analyse system performance.

Introduction

In most industrial processes, there are possibilities when the temperature needs to be regulated automatically. For example, the temperature of fluid inside a reactor has to be controlled within a certain range in order to get a stable operation. Temperature control can be performed in two ways: through logic control, or fuzzy control. In logic control, the temperature is controlled by switching the heater on or off when a certain critical value is reached, whereas fuzzy control needs a true definition of "hot" before the control operation.

In this experiment, the logic control system is demonstrated. The controlled source temperature comes from the filament of a heater. The process is controlled by switching "ON" or "OFF" the voltage supply of the heater. When the filament temperature is higher than the controlled temperature, the Raspberry Pi will give a signal to switch off the heater. The heater will be switched on when the temperature of the filament is lower than the controlled temperature. The controlled temperature is a variable that can be specified by the user.

In this open-ended lab, students will design and implement a system to perform temperature control using a Raspberry Pi. This project challenges students to develop and build the necessary circuitry to process signals from a sensor trainer and interface it effectively with the Raspberry Pi. The goal is to create a complete system that captures the temperature sensor reading, processes the information, displays the temperature, and controls the heater. An overall block diagram for the measurement system is shown in Figure 8.1.

Experimental Equipment

1. Raspberry Pi 4B
2. Sensor trainer
3. Thermistor

4. Analog-to-digital converter, ADC0804
5. Operational amplifier, LM741
6. Relay
7. Resistors, capacitors (according to your design)
8. NPN transistor, 2N3904
9. LEDs
10. Power supply and connecting wires

Pending

Departure from the department's reference document: the official version of this experiment additionally specifies a DAC0800 (digital-to-analog converter), used to output two discrete levels representing the heater's ON/OFF state. This version deliberately omits the DAC0800 — switching a heater on or off is a binary decision, requiring only a single GPIO output bit (through the relay/BJT driver stage below), and no analog quantity needs to be synthesised. If your instructor requires the DAC0800 to be included, revert to the official department document for this section.

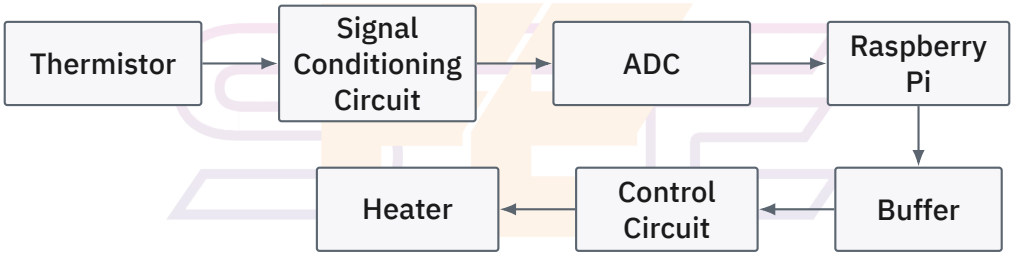

Figure 8.1: Block diagram of the temperature control system.

Component Notes

Thermistor: the thermistor is a type of resistor whose resistance varies with temperature. Its function is to convert the temperature to a certain value of resistance. From the given thermistor datasheet, you should be able to calculate the resistance as a function of temperature, and vice versa.

Signal conditioning element: used to convert the output of the thermistor into a form suitable for further processing — usually a DC voltage or current. A Wheatstone bridge and a potential divider are two examples of popular signal-conditioning circuits.

Voltage follower (buffer): the purpose of the buffer is to eliminate loading effects. Design your circuit based on the LM741 op-amp. You can also use the LM741 for your inverter and summing amplifier designs.

Analog-to-digital converter (ADC): an ADC is needed to translate the analogue input signal into a digital number so the Raspberry Pi can read it. The ADC0804 has a resolution

of 8 bits. Refer to its datasheet for operating information.

Procedure

1. Configure and construct the ADC in free-running mode, with the input connected to a potentiometer and the outputs to LEDs. Test your ADC.
2. Design and construct the signal-conditioning circuit. Test to verify its operation.
3. Configure the sensor trainer for thermistor ON/OFF control, using a BJT/relay driver stage between the Raspberry Pi's GPIO output and the heater terminal (Figure 8.2).
4. Switch on and start your experiment.
5. Set the controlled temperature, for example to 20°C, and record the observed thermostat temperature for 35 minutes using a 1-minute time interval (Table 8.1). Plot the temperature against time.
6. Repeat step 5 for 23°C and 25°C. Plot your results.
7. Discuss your findings.

Figure 8.2: ON/OFF control: BJT/relay driver stage between the Raspberry Pi GPIO and the heater terminal.

Table 8.1: Controlled vs. measured temperature log.

Controlled temp.	Time (minutes)	Measured temp. (°C)	Heater (ON/OFF)
20°C	1		
20°C	2		
20°C	...		
20°C	35		

Deliverables

1. **Code:** a programming script that accurately measures and displays the temperature based on sensor inputs, and controls the heater accordingly.
2. **System setup diagram:** a diagram showing the placement and wiring of sensors to the Raspberry Pi.
3. **Documentation:** a report that explains the setup, the temperature control process, sample readings, discussion, and any challenges encountered — including your justification for omitting the DAC0800 (see Experimental Equipment).
4. **Demo:** a short presentation demonstrating the working system and the collected data.

Evaluation Criteria

(a) Performance of temperature control:

- *Accuracy of temperature control:* evaluate the system's ability to maintain the desired temperature setpoint within an acceptable margin of error (e.g. ± 1°C).
- *Response time:* measure how quickly the system adjusts the heater to reach and stabilise at the target temperature after a disturbance or change in setpoint.
- *Stability:* assess whether the temperature remains stable without significant oscillations or overshoots during operation.
- *Range of operation:* verify the system's ability to control temperature over the required range (e.g. 0°C to 100°C).

(b) Code functionality: proper functioning of the program without errors.

(c) Creativity in design: unique solutions for circuit design, HMI, control algorithm, code optimisation, or data handling.

(d) Clarity of documentation: clear and thorough explanation of the approach, results, and challenges.